You said:

我想要实现一下内容，请帮我规划相应的设计方案，输出为markdown格式：使用Three.js、HTML、CSS和TypeScript开发完整、可交互的实验室级化学气相沉积法（CVD）生长二维材料设备的拆解可视化网页。所有几何模型与场景元素均由代码程序化生成，不依赖任何外部3D模型文件。场景需精准还原CVD设备外观及核心部件。交互支持：点击部件触发平滑拆解/复原动画、鼠标自由旋转与缩放视角、点击或悬停查看部件详情。视觉采用深色科技感背景，部件拆解时带发光高亮效果，并以标注显示各部件名称与关键参数。页面包含部件清单面板、参数说明区域与操作引导提示。输出为可直接在浏览器运行的单页应用。

ChatGPT said:

下面给出一份可以直接用于项目立项、技术开发和后续 AI 编程实施的 Markdown 设计方案。方案以 **Three.js + TypeScript + HTML + CSS** 为核心，目标是实现一个真正具有三维交互、程序化建模、部件拆解动画和参数展示能力的实验室级 CVD（Chemical Vapor Deposition，化学气相沉积）设备可视化单页应用。

设计上需要特别注意一点：CVD 设备存在多种结构形式。本方案以**水平式管式炉 CVD 系统**为基础，包含石英反应管、加热炉、气体输送管路、质量流量控制器、气源、真空系统及控制单元，并为后续扩展低压 CVD（LPCVD）和其他二维材料生长系统预留接口。

实验室级 CVD 二维材料生长设备拆解可视化网页设计方案

一、项目概述

1.1 项目目标

开发一个无需外部 3D 模型文件、可以直接在浏览器中运行的单页 Web 应用，通过程序化三维建模与交互动画，展示实验室级 CVD 设备的整体结构、关键组件、气体流路及工作原理。

核心目标包括：

- **设备可视化：** 使用 Three.js 程序化构建完整的 CVD 设备及其关键零部件。
- **交互式拆解：** 点击任意支持拆解的部件，使其沿预定义方向平滑移动，实现拆解与复原。
- **自由观察：** 支持鼠标旋转、平移、缩放，以及预设视角快速切换。
- **部件信息展示：** 点击或悬停部件时，显示名称、功能、结构说明及关键参数。
- **科技感界面：** 使用深色背景、冷色发光轮廓、精细标注线和半透明信息面板。
- **教学与演示：** 提供部件清单、拆解控制、参数说明、操作提示和设备工作流程说明。

1.2 设计原则

设计维度	设计要求
建模方式	全部三维几何体由代码生成，不加载外部模型文件
交互方式	鼠标拾取、点击、悬停、动画及面板联动
动画效果	平滑拆解、复原、发光高亮和标注跟随
工程结构	TypeScript 模块化开发，职责清晰，便于扩展
运行方式	本地静态 Web 服务启动，无需后端服务
科学准确性	区分设备结构示意、实际规格和可配置参数
性能要求	控制几何复杂度、材质数量及实时渲染开销

1.3 预期成果

最终交付一个可直接在浏览器中运行的单页应用，建议以 Vite 作为开发和构建工具。开发阶段运行本地服务器，生产阶段输出静态 HTML、CSS、JavaScript 等文件。

应用应实现完整的可交互三维场景，而不只是展示一张渲染图片。

二、技术选型与总体架构

2.1 技术栈

技术	推荐方案	主要职责
3D 引擎	Three.js	三维场景、几何体、材质、灯光、渲染
开发语言	TypeScript	类型约束、组件接口、业务逻辑
页面结构	HTML5	页面布局与信息区域
页面样式	CSS3	深色科技风、响应式布局、发光效果
构建工具	Vite	开发服务器、模块加载、生产构建
相机控制	OrbitControls	旋转、缩放、平移
动画系统	Three.js AnimationMixer 或自定义 Tween	拆解与复原动画
几何建模	Three.js 原生 Geometry	管路、炉体、阀门、法兰及设备外壳
标注系统	HTML Overlay + Three.js 投影	屏幕空间标签、引线、参数注释

图标	内联 SVG 或 CSS 绘制	按钮、状态图标、面板装饰

建议采用 Three.js 原生几何体完成核心建模。对于本项目的规则机械结构， `BoxGeometry` 、 `CylinderGeometry` 、 `SphereGeometry` 、 `TorusGeometry` 、 `LatheGeometry` 、 `TubeGeometry` 和自定义 `BufferGeometry` 已经足以构建具有辨识度的设备。

不需要引入 React 或 Vue。对于单页面、以三维场景为中心的应用，使用原生 TypeScript 模块可以减少依赖和额外的状态管理复杂度。

2.2 系统架构

整个应用分为五个逻辑模块：

1. **SceneManager：** 管理场景、相机、渲染器、灯光和渲染循环。
2. **CVDModelBuilder：** 生成所有设备几何体及其层级关系。
3. **InteractionManager：** 处理鼠标拾取、悬停、点击和视角控制。
4. **ExplodeController：** 管理部件拆解、复原、动画插值及状态切换。
5. **UIController：** 负责部件清单、详情面板、参数展示、标签和操作提示。

核心设计原则是：**三维模型负责空间表现，业务控制器负责交互逻辑，UI 层负责信息展示，三者通过统一的部件数据模型连接。**

三、CVD 设备三维建模设计

3.1 设备结构组成

建议将设备分为八个主要子系统，每个子系统包含多个可以独立选择的零部件。



01. 加热炉与炉体组件

- 炉体外壳、炉门、隔热层和加热区域。
- 内部石英反应管及炉膛空间。
- 炉体支架、底座与固定结构。
- 支持外壳分离、炉体轴向拆解和内部结构查看。



02. 石英反应管与密封组件

- 石英管、端部法兰、密封圈和接头。
- 反应区、基底舟及衬底位置。
- 气体入口与出口连接结构。
- 支持沿管轴方向拆解，并展示内部结构。



03. 气体输送系统

- 气瓶、减压阀、压力表、气体管路。
- 质量流量控制器（MFC）及切换阀。
- 气体混合节点、进气歧管和连接接头。
- 可用不同颜色区分不同气体支路。



04. 真空与排气系统

- 真空泵、排气管路和真空接口。
- 压力传感器、隔离阀和节流阀的示意结构。
- 支持与反应管分离，展示气体出口路径。

其余四个子系统如下：

子系统	主要建模对象	建模重点
05. 温控系统	热电偶、炉温传感器、温控器、连接线	展示测温位置与控制关系
06. 样品与基底	基底舟、衬底片、粉末源或固态前驱体容器	展示材料放置位置与反应区
07. 机架与防护结构	底板、立柱、横梁、固定夹具、防护罩	作为设备空间布局和拆解基准
08. 控制与监测单元	控制箱、显示屏、旋钮、接口和状态灯	展示温度、流量、压力等监测项目

注意： 这是可扩展的参考结构，不意味着所有实验室 CVD 系统都具备上述全部部件。气源数量、压力控制方式、真空泵类型及炉体结构应作为配置项，而不是写死在模型中。

3.2 三维场景的坐标系统

建议统一采用以下坐标约定：

- X 轴：反应管轴向，定义为主要拆解方向。
- Y 轴：设备左右方向。
- Z 轴：设备高度方向。
- 原点：加热炉与反应管中心的交汇位置。
- 单位：内部使用统一的模型单位，推荐按毫米级比例组织，必要时整体缩放到适合相机观察的尺寸。

示例尺寸仅用于确定视觉比例，不代表特定厂商设备的真实规格：

对象	示例尺寸
设备底座	1200 × 650 × 70
加热炉外壳	650 × 360 × 360
石英反应管	长 1100，外径 60
石英管壁厚	3
控制箱	260 × 180 × 350

实际设备参数应允许用户通过配置文件调整。尤其是管径、炉体长度、法兰尺寸和支架位置，应由同一份配置数据驱动，避免修改某个部件后出现装配错位。

3.3 程序化几何建模方法

几何对象	推荐实现方式
机架、炉体、控制箱	`BoxGeometry`
管道、接头、阀体	`CylinderGeometry` 、 `TubeGeometry`
法兰、密封圈	`CylinderGeometry` 、 `TorusGeometry`
弯曲管路	`CatmullRomCurve3` + `TubeGeometry`
压力表	圆柱、圆盘、指针组合
螺栓与螺母	圆柱、六边形自定义几何体

石英管	半透明圆柱壳体或自定义空心管几何体
加热丝	螺旋曲线 + `TubeGeometry`
基底舟	自定义 `BufferGeometry` 或组合几何体
电控屏幕	平面几何体 + CanvasTexture
设备标签	HTML 元素投影或 `CSS2DRenderer`

石英管应尽量采用具有内外壁结构的空心管模型，而不是单一的透明实心圆柱。这样可以更清楚地表现管壁、内部空间和基底位置。

加热炉可以采用“外壳 + 隔热层 + 加热区 + 反应管”的嵌套结构。炉体外壳支持隐藏或拆解后，可以展示内部构造。

3.4 材质与外观设计

金属外壳

深灰色金属、适度粗糙度、边缘高光。

石英玻璃

低饱和蓝白色半透明效果，适度反射和边缘高亮。

加热区域

橙红色发光材质，表示高温区域；不将发光效果误认为实际温度测量。

选中和拆解高亮

青色轮廓、柔和发光及辅助标注线。

推荐使用 `MeshStandardMaterial` 或 `MeshPhysicalMaterial` 实现主要材质，避免所有部件都使用高反光金属材质。

透明石英玻璃应注意深度写入、渲染顺序和双面显示等问题。对于高亮，可以使用 Emissive 材质配合 `OutlinePass`，或采用轻量级边缘线框方案；不要简单地为所有零部件增加大量实时点光源。

四、交互式拆解系统设计

4.1 拆解与复原机制

每个可交互部件都应拥有唯一的 `id`、对应的三维对象、初始位置、拆解位置、显示名称和交互状态。

拆解不能简单地将模型随机移动，而应根据设备的装配关系，预先定义每个组件的拆解方向和目标位置。

例如：

- 炉体外壳：沿 Z 轴向上移动，露出内部炉膛。
- 石英反应管：沿 X 轴向外抽出。
- 气体管路：沿指定方向偏移，避免与其他部件重叠。
- MFC 模块：沿 Y 轴向外移出。
- 控制箱：沿 Z 轴向上或沿 Y 轴侧向移出。
- 基底舟：沿 X 轴从反应管中抽出。
- 密封法兰：沿 X 轴向外移动，展示端部结构。

拆解距离、方向和顺序应配置化。不同设备型号不应依赖一套固定的拆解坐标。

4.2 部件状态机

建议使用以下状态：

状态	说明
`assembled`	完整装配状态
`exploding`	正在拆解
`exploded`	已完全拆解
`assembling`	正在复原

状态切换规则：

动画中途可以接受新的用户操作，但必须取消或替换旧动画，防止两个动画同时修改同一对象的位置。

4.3 动画实现

推荐使用 `requestAnimationFrame` 配合自定义插值函数，或者引入轻量级补间动画库。对于单个单页应用，也可以直接通过 Three.js 渲染循环更新动画状态。

位置插值公式：

$$\mathbf{p}(t) = \mathbf{p}_0 + E(t)\,(\mathbf{p}_1 - \mathbf{p}_0)$$

其中：

- $\mathbf{p}_0$：动画起始位置。
- $\mathbf{p}_1$：动画目标位置。
- $E(t)$：缓动函数，输入归一化时间 $t \in [0, 1]$。

推荐使用 `easeInOutCubic`，让部件起步和停止时更加自然。

除平移外，还可以按需要增加少量旋转，但法兰、管路和玻璃反应管等部件应以符合机械装配逻辑的运动为主。

4.4 三种拆解操作模式

模式	行为	适用场景
单部件拆解	只移动选中的部件	查看单个组件结构
子系统拆解	同时移动某个子系统的多个部件	展示气路或炉体内部
全局爆炸视图	按预设偏移量展开所有可拆解部件	设备整体结构讲解

全局拆解时，建议对部件进行分层布局，避免所有组件沿相同方向平移后相互遮挡。

4.5 部件高亮机制

高亮效果建议包含三个层次：

1. **悬停高亮：** 鼠标经过时，部件颜色轻微变化。
2. **选中高亮：** 点击后显示明显的轮廓、名称和参数。
3. **拆解高亮：** 部件正在运动时出现短暂的青色或蓝色发光效果，动画结束后降低亮度。

可以通过 OutlinePass 实现轮廓高亮，使用 MeshStandardMaterial.emissive 实现局部发光，并通过少量动态线条辅助表现拆解轨迹。

注意，发光效果应只作用于相关部件，不能遮盖透明玻璃或影响对内部结构的观察。

五、三维视角与鼠标交互

5.1 相机控制

使用 PerspectiveCamera 和 OrbitControls 实现自由观察。

需要支持：

- 鼠标左键拖动：旋转视角。
- 鼠标滚轮：缩放。
- 鼠标右键拖动：平移视角。
- 双击部件：将相机目标平滑移动到该部件附近。
- 点击快捷按钮：切换预设视角。
- 点击“重置视角”：恢复初始相机位置。

推荐提供以下预设视角：

- 整体等轴测视图。
- 正视图。
- 俯视图。
- 侧视图。
- 炉体内部观察视图。

视角切换应使用相机位置和观察目标的平滑插值，而不是直接修改相机坐标造成跳变。

5.2 射线拾取

使用 THREE.Raycaster 识别鼠标所指向的部件。

建议在创建模型时，将每个可交互网格注册到统一的部件管理器。鼠标拾取后，通过对象层级向上查找对应的部件 ID，而不是依赖网格名称进行字符串判断。

伪代码：

```
const raycaster = new THREE.Raycaster();
const pointer = new THREE.Vector2();

function pickPart(
  event: PointerEvent,
```

```
  camera: THREE.Camera,
  canvas: HTMLCanvasElement
) {
  const rect = canvas.getBoundingClientRect();

  pointer.x = ((event.clientX - rect.left) / rect.width) * 2 - 1;
  pointer.y = -((event.clientY - rect.top) / rect.height) * 2 + 1;

  raycaster.setFromCamera(pointer, camera);

  const hits = raycaster.intersectObjects(
    interactiveObjects,
    true
  );

  return hits.length ? resolvePart(hits[0].object) : null;
}
```

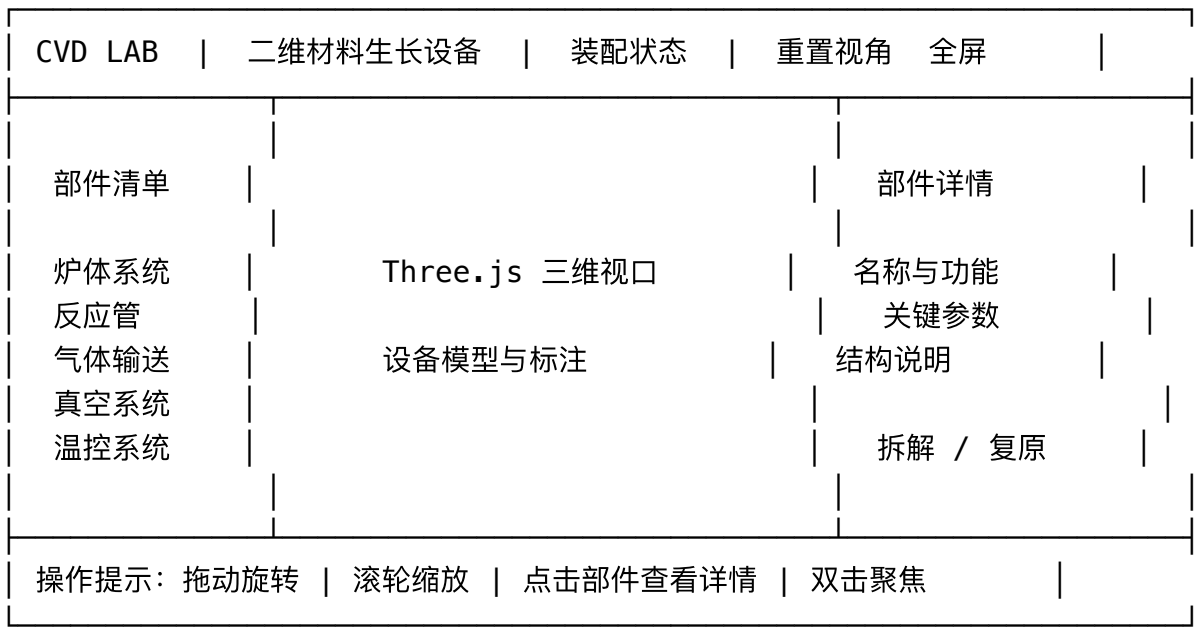
其中 `resolvePart()` 负责从被命中的网格向上查找已注册的部件对象。

鼠标悬停时更新高亮状态；点击时更新选中部件，并同步刷新左侧清单和右侧详情区域。

六、页面 UI 与视觉布局

6.1 整体布局

推荐使用 16:9 桌面端优先的布局，页面分为顶部工具栏、左侧部件清单、中间三维视口、右侧详情面板和底部操作提示。



6.2 左侧部件清单面板

建议使用可折叠树形结构：

```
设备结构
├── 炉体系统
│   ├── 炉体外壳
│   ├── 隔热层
│   └── 加热组件
├── 反应系统
│   ├── 石英反应管
│   ├── 进气法兰
│   ├── 排气法兰
│   └── 基底舟
├── 气体输送系统
│   ├── 气瓶
│   ├── 减压阀
│   ├── MFC
│   └── 气体管路
├── 真空系统
│   ├── 真空泵
│   └── 压力传感器
└── 控制系统
    ├── 温控器
    └── 控制箱
```

每个条目应具有以下能力：

- 点击条目：选中并高亮三维部件。
- 再次点击：根据当前状态拆解或复原。
- 展开子项：查看子系统下属组件。
- 当前选中项：显示醒目的状态标识。
- 已拆解部件：显示独立的状态图标。

6.3 右侧参数与详情区域

建议将信息划分为四个区块：

基本信息

- 部件名称。
- 部件编号。
- 所属子系统。

- 当前状态。

关键参数

- 几何尺寸。
- 材料。
- 工作温度范围。
- 压力或流量范围。
- 接口形式。

工作原理

- 部件在 CVD 工艺中的作用。
- 与其他部件的连接关系。
- 相关气体或热量传递路径。

交互操作

- 拆解当前部件。
- 复原当前部件。
- 聚焦当前部件。
- 隐藏或显示当前部件。

所有参数都应来自部件数据配置，不应在 HTML 中为每个部件单独硬编码。

6.4 标注系统

标注由三部分组成：

1. 部件名称标签。
2. 指向部件的引线。
3. 关键参数文字。

推荐采用 HTML Overlay 方案，或者使用 Three.js 的 `CSS2DRenderer` 进行屏幕空间标注。

标注锚点应来自三维模型中的局部坐标，通过 `Vector3.project(camera)` 转换为屏幕坐标。

标注系统应具备：

- 相机旋转时自动更新位置。
- 拆解时跟随对应部件移动。
- 遮挡时可隐藏次要标签。
- 标签过密时优先显示选中部件和关键部件。
- 标签超出屏幕时自动隐藏或限制位置。

默认视图只显示关键部件标签，选中某个子系统后再显示更多详细参数，避免界面过于拥挤。

6.5 深色科技风 UI 规范

建议使用以下基础配色：

用途	色彩建议
页面背景	#080D16
面板背景	#111A28
面板边框	#26374B
主强调色	#36D9E8
次强调色	#648BFF
普通文字	#E4EDF8
辅助文字	#8DA2B9
高温区域	#FF9B52
警示状态	#FF657A

UI 视觉效果可以使用渐变、半透明面板、细边框、微弱外发光和精细分隔线。

但不建议每个面板都使用强烈的霓虹效果。视觉焦点应始终落在三维设备上，文字应保持清晰可读。

七、部件数据模型设计

为了让模型、交互和 UI 共享同一份数据，建议建立统一的 TypeScript 类型定义。

7.1 部件类型

```
export type PartStatus =
  | "assembled"
  | "exploding"
  | "exploded"
  | "assembling";

export interface PartParameters {
  dimensions?: string;
  material?: string;
  temperatureRange?: string;
```

```
  pressureRange?: string;
  flowRange?: string;
  interfaceType?: string;
}

export interface CVDPartConfig {
  id: string;
  name: string;
  category: string;
  description: string;
  function: string;

  parentId?: string;
  children?: string[];

  initialPosition: [number, number, number];
  explodeOffset: [number, number, number];

  parameters: PartParameters;
  color: string;

  interactable: boolean;
  initiallyVisible: boolean;
}
```

7.2 配置示例

```
export const cvdParts: CVDPartConfig[] = [
  {
    id: "furnace-shell",
    name: "加热炉外壳",
    category: "加热系统",
    description: "用于容纳并保护加热组件及隔热结构。",
    function: "提供受控的高温反应环境。",
    initialPosition: [0, 0, 0],
    explodeOffset: [0, 0, 3.5],
    parameters: {
      dimensions: "650 × 360 × 360（示意）",
      material: "金属外壳",
      temperatureRange: "按设备规格配置"
    },
    color: "#657386",
    interactable: true,
    initiallyVisible: true
```

```
  },
  {
    id: "quartz-tube",
    name: "石英反应管",
    category: "反应系统",
    description: "为前驱体输运和薄膜生长提供反应空间。",
    function: "承载气相反应环境并容纳样品。",
    initialPosition: [0, 0, 0],
    explodeOffset: [4.5, 0, 0],
    parameters: {
      dimensions: "长度 1100、外径 60（示意）",
      material: "石英玻璃",
      pressureRange: "按设备配置确定"
    },
    color: "#9FCBD8",
    interactable: true,
    initiallyVisible: true
  }
];
```

这里的坐标和参数仅用于演示配置结构。实际项目中，应将模型尺寸、初始位置和拆解偏移统一在一套设备装配配置中管理，并根据父子层级处理局部坐标和世界坐标，避免重复应用位移。

另外，真实设备的工作温度、压力范围、管径及接口尺寸必须根据目标设备的规格资料确定，不能把示意参数当成实验设备的真实额定参数。

八、项目目录结构

推荐采用以下目录，便于分阶段开发和后期维护。

```
cvd-visualizer/
├── index.html
├── package.json
├── tsconfig.json
├── vite.config.ts
├── public/
│   └── favicon.svg
├── src/
│   ├── main.ts
│   ├── styles/
│   │   ├── base.css
│   │   ├── layout.css
│   │   ├── panels.css
│   │   └── effects.css
```

```
│   ├── core/
│   │   ├── SceneManager.ts
│   │   ├── CameraController.ts
│   │   └── RenderLoop.ts
│   ├── models/
│   │   ├── CVDModelBuilder.ts
│   │   ├── FurnaceBuilder.ts
│   │   ├── QuartzTubeBuilder.ts
│   │   ├── GasSystemBuilder.ts
│   │   ├── VacuumSystemBuilder.ts
│   │   ├── ControlSystemBuilder.ts
│   │   └── GeometryFactory.ts
│   ├── interaction/
│   │   ├── InteractionManager.ts
│   │   ├── PartRegistry.ts
│   │   ├── ExplodeController.ts
│   │   ├── HighlightController.ts
│   │   └── CameraAnimator.ts
│   ├── ui/
│   │   ├── UIController.ts
│   │   ├── PartListPanel.ts
│   │   ├── PartDetailPanel.ts
│   │   ├── AnnotationManager.ts
│   │   └── Toolbar.ts
│   ├── data/
│   │   ├── cvdParts.ts
│   │   └── equipmentConfig.ts
│   └── utils/
│       ├── easing.ts
│       └── coordinates.ts
└── README.md
```

其中， `GeometryFactory.ts` 封装常用机械结构， `CVDModelBuilder.ts` 负责装配整个设备， `PartRegistry.ts` 统一登记可交互对象， `ExplodeController.ts` 管理动画， `UIController.ts` 负责协调界面更新。

所有核心模型都通过 TypeScript 代码生成，不需要 `GLTF` 、 `OBJ` 、 `FBX` 等外部三维模型文件。项目可以保留纯代码几何建模，同时使用 CSS、SVG 和 Canvas 绘制界面与图形。

九、核心功能实现流程

9.1 应用初始化

应用启动后依次完成：

1. 创建 Three.js 场景、相机和渲染器。
2. 设置环境光、方向光、辅助灯光和背景。
3. 初始化 OrbitControls。
4. 根据设备配置生成底座、炉体、反应管及各个子系统。
5. 将三维部件注册到统一的部件管理器。
6. 创建部件清单、详情面板和标注层。
7. 绑定鼠标、键盘及 UI 事件。
8. 启动动画与渲染循环。

9.2 点击部件后的数据流

建议将部件选中状态和动画状态分开管理。一个部件可以在拆解后仍然保持选中状态，也可以在动画结束后继续显示其详情。

9.3 核心拆解控制器示例

以下示例展示拆解动画的基本实现方式。实际项目需要进一步处理子部件、父子层级、动画中断及状态同步。

```
import * as THREE from "three";

interface ExplodablePart {
  object: THREE.Object3D;
  assembledPosition: THREE.Vector3;
  explodedPosition: THREE.Vector3;
```

```
  progress: number;
  targetProgress: number;
}

export class ExplodeController {
  private parts: ExplodablePart[] = [];
  private duration = 900;

  register(
    object: THREE.Object3D,
    explodeOffset: THREE.Vector3
  ) {
    const assembledPosition = object.position.clone();

    this.parts.push({
      object,
      assembledPosition,
      explodedPosition: assembledPosition
        .clone()
        .add(explodeOffset),
      progress: 0,
      targetProgress: 0
    });
  }

  explodeAll() {
    for (const part of this.parts) {
      part.targetProgress = 1;
    }
  }

  assembleAll() {
    for (const part of this.parts) {
      part.targetProgress = 0;
    }
  }

  setExploded(id: number, exploded: boolean) {
    const part = this.parts[id];
    if (part) {
      part.targetProgress = exploded ? 1 : 0;
    }
  }

  update(deltaMs: number) {
```

```
    const step = Math.min(deltaMs / this.duration, 1);

    for (const part of this.parts) {
      const diff = part.targetProgress - part.progress;

      if (Math.abs(diff) < 0.001) {
        part.progress = part.targetProgress;
      } else {
        part.progress += diff * step;
      }

      const t = part.progress;
      const eased = t * t * (3 - 2 * t);

      part.object.position.lerpVectors(
        part.assembledPosition,
        part.explodedPosition,
        eased
      );
    }
  }
}
```

使用时，在渲染循环中传入帧间隔：

```
const clock = new THREE.Clock();

function animate() {
  requestAnimationFrame(animate);

  const deltaMs = clock.getDelta() * 1000;

  explodeController.update(deltaMs);
  controls.update();
  renderer.render(scene, camera);
}

animate();
```

这个示例使用渐进插值展示原理。正式版本建议使用时间归一化的动画轨迹，确保不同帧率下的动画持续时间一致；同时根据 id 而不是数组索引查找部件，并对父子部件和局部坐标进行专门处理。

十、性能优化与工程可靠性

虽然项目不加载外部模型，但程序化建模同样可能产生大量网格和材质，需要主动进行性能管理。

10.1 渲染优化

- 重复的螺栓、螺母和相同结构尽可能使用 `InstancedMesh`。
- 合并不需要独立交互的静态装饰几何体。
- 共享材质和几何体，避免重复创建。
- 限制设备内部的小型零件数量，优先保证整体结构清晰。
- 通过 `renderer.setPixelRatio()` 控制高分辨率设备的渲染开销。
- 页面失去焦点时，可以降低渲染频率。
- 正确释放临时几何体、材质、纹理及后期处理资源。

10.2 透明材质与后期处理

石英管、观察窗及透明保护罩需要谨慎处理透明度。过多透明对象容易出现深度排序问题，建议：

- 限制同时叠加的透明层数。
- 必要时使用独立内外壁网格。
- 根据观察需求调整 `depthWrite`。
- 为玻璃设置适当的 `roughness` 和 `metalness`。
- 后期处理只对必要对象进行高亮，避免过强的 Bloom 效果。

10.3 响应式布局

桌面端优先保证完整的三栏布局；在较窄屏幕下，建议将左右面板改为可展开抽屉。

三维视口尺寸发生变化时，应同步更新相机宽高比、投影矩阵和渲染器尺寸。面板折叠、全屏切换及窗口变化都应触发对应的布局更新。

十一、分阶段开发计划

建议分为六个阶段，先完成可运行的核心功能，再逐步提升结构细节和视觉表现。

阶段	主要任务	验收标准
第一阶段：基础框架	Vite、TypeScript、Three.js 初始化；深色页面与相机控制	页面可运行，设备区域可旋转、缩放
第二阶段：核心建模	炉体、石英管、底座、基底舟及主要管路	设备整体结构清晰，所有模型由代码生成

第三阶段：拆解系统	部件注册、单件拆解、复原、全局拆解	点击部件可平滑拆解和复原
第四阶段：信息交互	射线拾取、部件清单、详情面板、参数标签	三维模型与 UI 状态保持同步
第五阶段：视觉完善	材质、发光高亮、视角预设、标注优化	达到深色科技风视觉目标
第六阶段：测试与交付	性能优化、响应式适配、异常处理、构建	可通过生产构建并在浏览器运行

推荐的开发顺序是先完成第一至第三阶段，再开发 UI 联动。不要一开始就制作复杂的透明材质、发光后期处理和大量精细零件，否则很容易陷入视觉细节开发，而核心交互仍然不完整。

十二、测试与验收清单

正式交付前，应逐项检查以下功能。

三维场景

- ○ 所有主要部件均由程序化几何体生成。
- ○ 无需外部 3D 模型文件即可构建场景。
- ○ 整体布局符合水平式管式 CVD 的基本结构。
- ○ 相机旋转、缩放、平移和视角重置正常。
- ○ 窗口尺寸变化时场景不会变形。

拆解与复原

- ○ 单个部件能够独立拆解与复原。
- ○ 支持全局拆解和全局复原。
- ○ 动画平滑，没有明显位置跳变。
- ○ 动画中途可以安全切换方向。
- ○ 拆解后部件之间尽量避免严重重叠。
- ○ 子部件不会因父对象移动而产生意外位移。

信息与界面

- ○ 悬停部件能够触发高亮。
- ○ 点击部件能够显示对应详情。
- ○ 部件清单与三维选中状态同步。
- ○ 参数标签能够跟随部件移动。
- ○ 参数缺失时显示合理的占位内容。
- ○ 所有按钮均具有实际功能。

- [ ] 页面在不同窗口尺寸下保持可用。

工程与交付

- [ ] TypeScript 类型检查通过。
- [ ] Vite 生产构建成功。
- [ ] 控制台无持续性错误。
- [ ] 长时间交互后无明显性能下降。
- [ ] 所有资源在组件或应用销毁时得到正确释放。
- [ ] README 中包含启动、构建和操作说明。

十三、最终交付形式

建议最终提供以下文件：

1. `index.html`：应用入口。
2. `src/`：全部 TypeScript、CSS 和程序化建模代码。
3. `package.json`：依赖与运行脚本。
4. `README.md`：安装、启动、操作和构建说明。
5. `dist/`：Vite 生成的生产构建文件。

本地开发可使用：

```
npm install
npm run dev
```

生产构建可使用：

```
npm run build
npm run preview
```

需要注意，Vite 开发服务器和预览服务器并不是将源码直接双击打开的方式。若希望用户下载后直接打开一个 HTML 文件就能使用，可以在交付阶段进一步配置单文件打包方案，并确认 Three.js、CSS 和所有运行时依赖都已正确内嵌。另一种更稳定的方式是交付一个可部署的静态网站目录。

最终验收目标： 用户打开网页后，可以看到完整的程序化 CVD 三维设备，通过鼠标自由观察设备，点击任意主要部件查看详情，并通过拆解、复原和子系统操作理解设备的内部结构。所有演示参数均有明确标识，不将示意值误认为真实设备的技术规格。

十四、建议优先确定的两个设计方向

为了让后续开发更加顺利，建议优先确定模型精细度和交付方式。这两项会直接影响几何建模工作量、交互设计以及最后的打包策略。

你希望先采用哪种建模方案?

方案 A：教学演示型
优先完成完整设备、部件清单、拆解动画与原理说明，适合课堂演示，开发速度较快。

方案 B：高精度工程展示型
重点构建机械细节、连接关系、内部结构和参数标注，适合科研展示，开发工作量更大。